

Real-time noise-adapted quantum error correction decoding with zero training overhead

Sheng-Kai Huang^{1,*}

¹*Independent Researcher, Taiwan*

(Dated: March 19, 2026)

Noise-aware quantum error correction decoders improve logical error rates, but existing approaches carry steep deployment costs: IBM’s noise-learning protocol requires ~ 4000 training circuits and hours of classical post-processing; machine-learning decoders such as AlphaQubit demand millions of samples and GPU training. We show that Bayesian noise tracking via the retrodiction parameter $\tau = 1 - F$ achieves the same noise-informed improvement— $2.67\times$ reduction at code distance 7 under non-uniform noise, $14\times$ under biased noise—while deploying in minutes from standard gate-set characterization data, with zero additional training circuits. The retrodiction gap δD , computable from syndrome statistics alone, serves as a free health monitor that detects noise drift and triggers recalibration without extra measurement overhead. We present deployment cost comparisons across four noise-characterization methods and argue that the bottleneck for noise-adapted QEC on near-term hardware is not algorithmic accuracy but operational overhead.

INTRODUCTION

The case for noise-aware decoding in quantum error correction is settled. When a decoder’s assumed noise model matches reality, logical error rates drop. Tuckett *et al.* [9] showed threshold enhancements under biased noise. Hockings *et al.* [8] demonstrated systematic gains from noise-aware weight assignment on surface codes. Google’s AlphaQubit [10] used recurrent neural networks trained on millions of syndrome samples to achieve state-of-the-art logical error rates on the Willow processor [?]. Joshi *et al.* [7] demonstrated break-even performance with noise-adapted Petz recovery on a trapped-ion processor.

The question is no longer whether noise-informed decoding helps. It does, and the improvement factors—ranging from $1.5\times$ to $14\times$ depending on noise structure—are well characterized. The question that matters for deployment is: *at what cost?*

Consider the operational budget. IBM’s noise-learning protocol [15?] runs ~ 4000 randomized circuits to fit a sparse Pauli-Lindblad model, requiring several hours of device time. AlphaQubit [10] trains on 10^6 – 10^7 syndrome samples with GPU compute measured in days. Gate-set tomography via pyGSTi [?] provides a complete noise characterization but at prohibitive circuit cost ($> 10^4$ circuits) that scales poorly with qubit count.

These costs recur whenever the noise drifts—which, on superconducting hardware, occurs on timescales of minutes to hours [?]. A noise-aware decoder is only as good as its noise model. If recalibration takes four hours, the decoder runs on stale weights most of the time.

We present an alternative approach based on Bayesian noise tracking. The retrodiction parameter $\tau_j = 1 - F_j$ for each gate j , derived from single-gate characterization data, feeds directly into MWPM edge weights via $w_j = \log[(1 - \tau_j)/\tau_j]$. The resulting decoder achieves the same logical error rate improvement as other noise-aware

methods—because the improvement depends on having accurate noise information, not on how that information was obtained. The difference is deployment cost: minutes instead of hours, zero training circuits beyond standard device characterization, and real-time adaptation to drift through continuous τ updates.

BACKGROUND: NOISE-INFORMED MWPM

The minimum-weight perfect matching (MWPM) decoder [11, 13] maps syndromes to corrections by finding the minimum-weight matching on a decoding graph. Each edge e carries a weight w_e related to the probability p_e of the corresponding error:

$$w_e = \log \frac{1 - p_e}{p_e}. \quad (1)$$

When p_e reflects the true noise, MWPM is near-optimal [12]. When p_e is wrong—uniform weights on non-uniform hardware, for instance—performance degrades.

This is the entire algorithmic content of noise-aware MWPM decoding: get the right p_e , plug into Eq. (1). There is no decoder-side innovation. The question reduces to noise characterization: *how do you measure p_e quickly, cheaply, and repeatedly?*

The retrodiction parameter $\tau = 1 - F(\rho, \mathcal{R}_\sigma \circ \mathcal{N}(\rho))$ provides a per-gate noise metric traceable through circuit depth via Bayesian composition [16]. For Pauli noise channels, τ_j maps directly to p_j (the per-qubit error probability), giving $w_j = \log[(1 - \tau_j)/\tau_j]$. On QuTech Tuna-9 hardware, Bayesian τ -tracking improves noise prediction by 46% at circuit depth 50 compared to independent gate models.

DEPLOYMENT COST COMPARISON

We compare four noise-characterization methods that feed noise-aware MWPM decoding (Table ??). All four produce the same type of output—per-edge error probabilities—and therefore achieve comparable logical error rates when the noise estimates are accurate. The differences are operational.

Several remarks are in order.

Same accuracy, different overhead.—The logical error rate improvement from noise-aware decoding depends on two things: the accuracy of the noise estimates and the degree of noise non-uniformity. Given equally accurate estimates, all four methods yield the same decoder weights and therefore the same logical error rate. The claim is not that τ -chrono decoding is more accurate. It is that τ -chrono achieves the same accuracy with less overhead.

Why zero training circuits?—The τ parameters are extracted from the same single- and two-qubit gate characterization data that hardware operators already collect during routine calibration. No additional circuits are run. For platforms that publish per-gate fidelities (IBM Quantum, Quantinuum, QuEra), the τ values are literally available in the device metadata.

Drift adaptation.—On superconducting hardware, T_1 and T_2 fluctuations cause noise drift on timescales of ~ 10 minutes [?]. A method requiring 4000 circuits for recalibration cannot keep pace. The τ -chrono approach updates weights from streaming characterization data or, as we show below, from syndrome statistics themselves, enabling continuous adaptation.

SURFACE CODE SIMULATIONS

We verify that τ -informed MWPM achieves the expected noise-aware improvements on rotated surface codes at distances $d = 3, 5, 7$, simulated with Stim [14] and decoded with PyMatching [13].

Non-uniform noise.—Thirty percent of data qubits are assigned $4\times$ higher error rate. The τ -informed decoder uses the true per-qubit rates; the baseline uses uniform weights. Improvement factors: $1.09\times$ ($d=3$), $1.23\times$ ($d=5$), $2.67\times$ ($d=7$) at $p_{\text{base}} = 0.001$, consistent with Hockings *et al.* [8]. The growth with code distance is expected: larger codes sample more of the noise heterogeneity.

Biased noise.—When Z errors dominate X errors by a factor η , a bias-aware decoder yields $1.87\times$ improvement at $\eta = 10$ and $14.2\times$ at $\eta = 100$, reproducing the threshold enhancement of Tuckett *et al.* [9]. The effective threshold shifts from $\sim 0.7\%$ to $\sim 37.6\%$.

Noise drift.—Hot qubits are randomly reassigned every 1000 rounds. A recalibrating decoder maintains logical error rate $\sim 5.1\%$; a stale decoder degrades to $\sim 6.6\%$, a

29% increase. The recalibrating decoder updates τ values every epoch at negligible computational cost (< 1 ms for weight recomputation on ~ 100 qubits).

These numbers are not new; they confirm the established literature. The point is that τ -chrono achieves them without dedicated training circuits.

THE RETRODICTION GAP AS A HEALTH MONITOR

The retrodiction gap

$$\delta D(\mathcal{R}) = D(\rho\|\sigma) - D(\mathcal{R} \circ \mathcal{N}(\rho)\|\mathcal{R} \circ \mathcal{N}(\sigma)) \quad (2)$$

quantifies how much information a recovery map $\mathcal{R}$ fails to restore after noise $\mathcal{N}$ acts. By the data processing inequality, $\delta D \geq 0$, with equality if and only if recovery is exact [1].

We verified on a 3-qubit bit-flip code that δD orders recovery maps consistently with entanglement fidelity F_e across four strategies spanning two orders of magnitude in fidelity (Table ??). The ordering holds at all tested noise levels $p \in [0.001, 0.2]$.

For deployment, the key property is that δD can be *estimated from syndrome statistics* without extra circuits. The reasoning is as follows. In a surface code with Pauli noise, the syndrome distribution encodes the marginal error rates $\{p_e\}$. Changes in the syndrome distribution—detectable via standard statistical tests on the syndrome bit strings already being collected—signal changes in the underlying $\{p_e\}$ and hence in δD .

Concretely, the syndrome entropy $H(S) = -\sum_s P(s) \log P(s)$ is monotonically related to the average error rate. A drift in $H(S)$ beyond a threshold (set by a Hoeffding bound on syndrome samples) triggers weight recalibration. This provides a *free* health monitor: the syndromes are measured regardless, and the entropy calculation is $O(1)$ per round.

Three caveats apply. First, the syndrome-based estimate of δD is approximate; it captures average drift but not correlated noise changes that leave marginals invariant. Second, the mapping from syndrome statistics to per-edge p_e values requires a noise model (e.g., independent Pauli errors); model misspecification limits the accuracy. Third, for general (non-Pauli) noise, the full δD computation requires knowledge of the complete noise channel, which is exponentially costly.

Within these limitations, syndrome-derived δD provides actionable drift detection at zero additional measurement cost.

WHAT THIS APPROACH DOES NOT DO

Honesty about scope matters more than broad claims. We state the limitations plainly.

TABLE I. Deployment cost comparison for noise-aware MWPM decoding on a distance-7 surface code (~ 100 data qubits). All methods produce per-edge noise estimates and achieve comparable logical error rate improvements when those estimates are accurate. The distinction is operational cost. “Circuits” counts quantum circuits executed beyond normal QEC operation. “Setup time” measures wall-clock time from cold start to a calibrated decoder. “Drift response” indicates how the method handles noise changes.

Method	Training circuits	Setup time	Drift response	Open source
τ -chrono (this work)	0 (uses existing char. data)	~ 10 min	continuous update	yes
IBM noise learning [15?]	~ 4000	~ 4 hr	periodic re-run	partial
pyGSTi [?]	$> 10^4$	~ 8 hr	full re-run	yes
AlphaQubit [10]	$10^6 - 10^7$ (training)	~ 24 hr (GPU)	retrain	no

No algorithmic advantage.—The τ -chrono decoder is a standard MWPM decoder with noise-informed weights. Any method that provides the same per-edge noise estimates will achieve the same logical error rate. The advantage is operational (speed, cost, adaptability), not algorithmic.

No advantage for uniform noise.—When hardware noise is spatially uniform and temporally stable, noise-aware and noise-unaware decoders perform identically. The gains reported here are proportional to the degree of noise heterogeneity.

Accuracy ceiling.—The τ values inherit the accuracy of the underlying gate characterization. If the characterization data are noisy or the noise model is misspecified, τ -chrono inherits those errors. It does not magically produce better noise estimates than the input data support.

Scalability of δD .—Computing δD exactly for a distance- d surface code requires manipulating $2^{O(d^2)}$ -dimensional operators. The syndrome-based proxy described above avoids this, but at the cost of approximation. This is a shared limitation of all information-theoretic decoder certificates.

DISCUSSION

The noise-aware decoding literature has focused on accuracy: how much do logical error rates improve? The answer—typically $2-15\times$ depending on noise structure—is well established [7–10]. What has received less attention is the deployment cost of obtaining and maintaining the noise model that enables these gains.

TABLE II. Retrodiction gap orders decoders consistently with fidelity. 3-qubit bit-flip code, $p = 0.05$.

Recovery	F_e	δD_{avg}
Majority vote	0.9928	0.0296
Petz recovery	0.9862	0.0504
Wrong correction	0.9025	0.0777
No correction	0.8574	0.0996

For near-term quantum computers running continuous QEC, the noise model must be refreshed on the same timescale as noise drift. A method that requires four hours of training is effectively a batch method; it cannot track drift on superconducting hardware where T_1 fluctuates on ten-minute timescales. The practical value of τ -chrono is that it closes this gap: the same noise-informed improvement, refreshed continuously, at negligible cost.

The retrodiction gap δD adds a diagnostic layer. Its connection to quantum relative entropy loss [3, 4] and to the Petz recovery map [1, 2] provides information-theoretic grounding, but its practical value is simpler: it is a number that goes up when the decoder’s noise model is getting stale. Tracked over time from syndrome statistics, it serves as a canary for recalibration—no extra circuits, no extra compute, no extra cost.

Whether this operational advantage matters depends on the use case. For one-shot experiments on stable hardware, any noise-characterization method suffices. For continuous QEC on drifting hardware—the regime that fault-tolerant quantum computing requires—deployment cost becomes the binding constraint.

Code and data are available at <https://github.com/akaiHuang/petz-recovery-unification>.

Hardware access was provided by QuTech through the Quantum Inspire platform. Numerical simulations used Stim [14] and PyMatching [13].

* akai@fawstudio.com

- [1] D. Petz, Sufficient subalgebras and the relative entropy of states of a von Neumann algebra, *Commun. Math. Phys.* **105**, 123 (1986).
- [2] A. J. Parzygnat and F. Buscemi, Axioms for retrodiction: achieving time-reversal symmetry with a prior, *Quantum* **7**, 1013 (2023).
- [3] M. Junge, R. Renner, D. Sutter, M. M. Wilde, and A. Winter, Universal recovery maps and approximate sufficiency of quantum relative entropy, *Ann. Henri Poincaré* **19**, 2955 (2018).
- [4] D. Sutter, M. Tomamichel, and A. W. Harrow, Strengthened monotonicity of relative entropy via pinched Petz

- recovery map, *IEEE Trans. Inform. Theory* **62**, 2907 (2016).
- [5] H. K. Ng and P. Mandayam, Simple approach to approximate quantum error correction based on the transpose channel, *Phys. Rev. A* **81**, 062342 (2010).
 - [6] I. Kim, Petz and Schumacher-Westmoreland recovery maps achieve optimal QEC threshold, arXiv:2603.06520 (2026).
 - [7] A. Joshi, M. Rudra, A. Dutta, S. Dhomkar, and P. Mandayam, Demonstrating noise-adapted quantum error correction with break-even performance, arXiv:2603.04564 (2026).
 - [8] J. Hockings, A. C. Doherty, and R. Harper, Noise-aware decoding for surface codes, arXiv:2502.21044 (2025).
 - [9] D. K. Tuckett, S. D. Bartlett, and S. T. Flammia, Ultra-high error threshold for surface codes with biased noise, *Phys. Rev. Lett.* **120**, 050505 (2018).
 - [10] Google DeepMind, Learning high-accuracy error decoding for quantum processors, *Nature* (2024).
 - [11] Google Quantum AI, Quantum error correction below the surface code threshold, *Nature* (2024).
 - [12] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, Topological quantum memory, *J. Math. Phys.* **43**, 4452 (2002).
 - [13] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, Surface codes: Towards practical large-scale quantum computation, *Phys. Rev. A* **86**, 032324 (2012).
 - [14] O. Higgott and C. Gidney, Sparse Blossom: correcting a million errors per core second with minimum-weight matching, arXiv:2303.15933 (2023).
 - [15] C. Gidney, Stim: a fast stabilizer circuit simulator, *Quantum* **5**, 497 (2021).
 - [16] E. van den Berg, Z. K. Mineev, A. Kandala, and K. Temme, Probabilistic error cancellation with sparse Pauli-Lindblad models on noisy quantum processors, *Nat. Phys.* **19**, 1116 (2023).
 - [17] Y. Kim *et al.*, Evidence for the utility of quantum computing before fault tolerance, *Nature* **618**, 500 (2023).
 - [18] E. Nielsen *et al.*, Gate set tomography, *Quantum* **5**, 557 (2021).
 - [19] S.-K. Huang, Petz recovery unification: $\tau = 1 - F$ as a universal noise metric for quantum circuits, GitHub/Zenodo (2026).